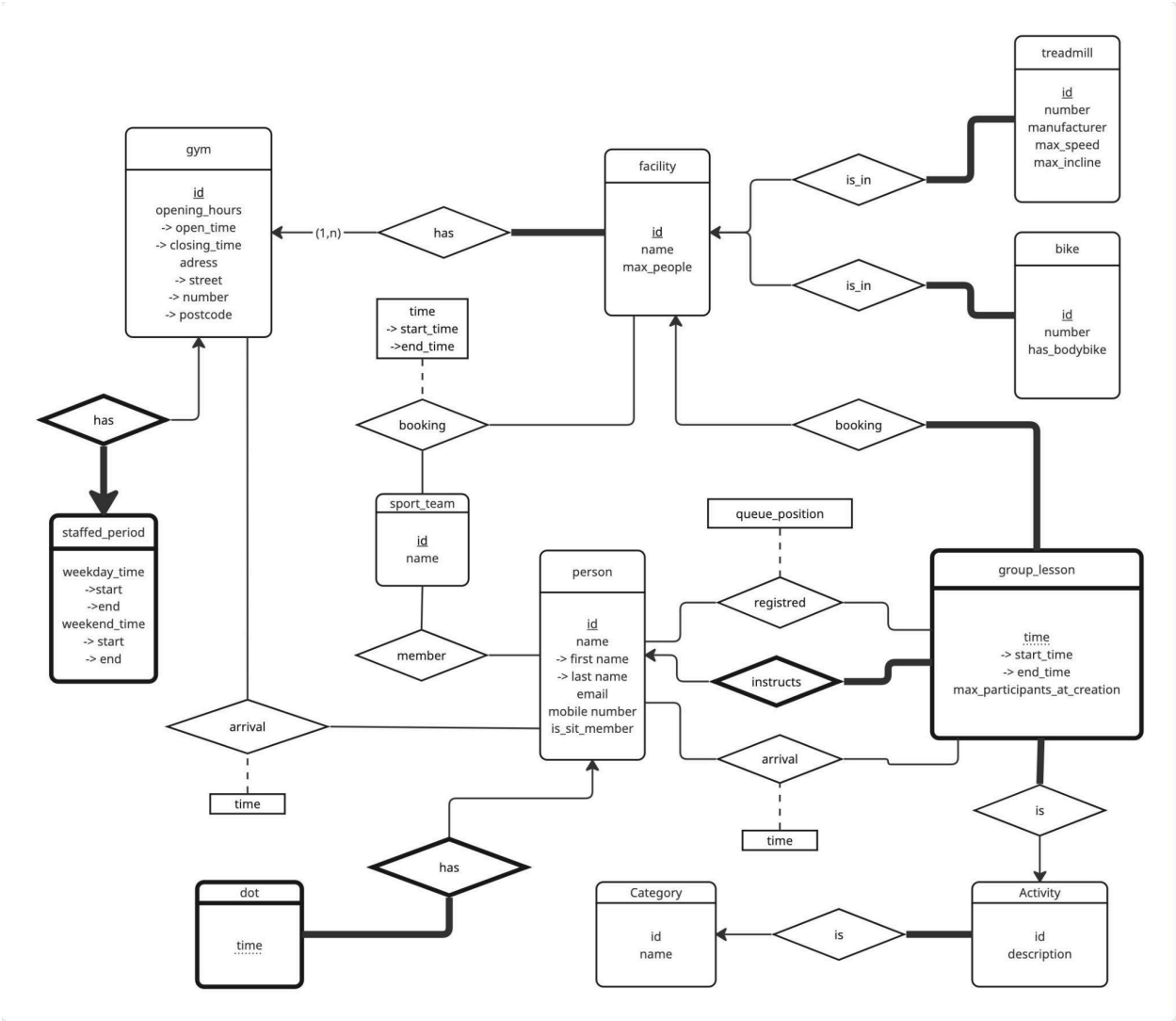
Group:
Brage Sebastian Brevik
Halvard Grimnes Haga
Group 151

ER DIAGRAM



Assumptions/explanations:
Facilities are all kinds of rooms one can find in the gym, for example a strength room, bike room, hall, toilet, shower, etc.

The activity and category entities exist to minimize the duplication of data, because each group lesson entity represents only a single instance of that group lesson, as the time values are date specific. Since a lot of group lessons are the same (Crossfit on Tuesdays for example) storing the same description multiple times would be inefficient.

Group lessons have the max_participants_at_creation time to preserve history. We need to store what the capacity at creation was, because if the capacity changes we don't want to lose historical data for how full the lessons were. For example, let's say a lesson has a capacity of 50 at creation and gets fully booked. If at a later date the facility gets upgraded and now has space for 100 people, then we don't want to have it look like only 50/100 people showed up to the lesson when gathering historical data for statistics.

Bikes and treadmills have an id as to not make them tied to their current facility. The alternative would have been to have them as weak entity sets connected to the facility, however this would make moving a bike/treadmill much more complicated, but with an id that isn't difficult at all.

We assume that the gyms don't change their opening/staffed periods from day to day, and instead follow the same system as SIT uses with specific times for weekdays and weekends.

Instructors are also people registered as a normal user, the only difference is that they partake in the "instructs" relation. And that by saying "this is shown by a first name of the person" we interpret this as when a user views a group lesson they only see the first name of the instructor (which is a person entity).

A person's arrival to a gym and group lessons are two separate things. This is because this is how SIT models it, you arrive at a gym and may then arrive at a group lesson afterwards. We need arrivals at gyms for statistics, and we need arrival at lessons for dots and also lesson specific statistics.

A person is not necessarily a SIT training member, as for example a person could be active in only a sports team, or they could have their membership paused.

Normal forms:
A relation is in BCNF if for all non-trivial functional dependencies X->Y, X is a superkey.
All relations are in BCNF:
gym: PK: id
All attributes depend only on id.
staffed_period: PK: gym_id
All attributes depend only on gym_id.
facility: PK: id
All attributes depend only on id.
treadmill: PK: id
All attributes depend on a superkey, either the PK or (facility_id, number)
bike: PK: id
All attributes depend only on a superkey, either the PK or (facility_id, number)
sport_team: PK: id
All attributes depend only on id
sport_team_member: PK: (sport_team_id, person_id)
No non-prime attributes and the prime attributes don't determine each other

sport_team_booking: PK: (sport_team_id, facility_id, start_time)
end_time is the only non-prime attribute and no subset of the PK determines end_time, but the PK as a whole does, and the prime attributes don't determine each other
person: PK: id
For all non-trivial functional dependencies they are dependent on either the PK or a different superkey like email or mobile_number
dot: PK: (person_id, time)
No non-prime attributes and the prime attributes don't determine each other
gym_arrival: PK: (gym_id, person_id, time)
No non-prime attributes and the prime attributes don't determine each other
group_lesson: PK: (start_time, instructor_id)
All non-trivial functional dependencies depend only on the superkey
group_lesson_arrival: PK: (person_id, group_lesson_start_time, group_lesson_instructor_id)
All non-trivial functional dependencies depend only on the superkey
category: PK: id
name is the only non-prime attribute and it depends entirely on the PK
activity: PK: id
All non-prime attributes depend only on the PK
group_lesson_registration: (person_id, group_lesson_start_time, group_lesson_instructor_id)
queue_position is the only non-prime attribute and it depends entirely on the PK, and the prime attributes don't determine each other

Restrictions handled by application or triggers:
A lot of the business logic not already implemented by the relational schema can be implemented with triggers in SQL or by the application layer. We consider triggers as outside of the schema and will therefore not include them, but business logic implemented by triggers or the application layer would be:

- Preventing registration after three dots (blacklisting)
- Adding dots for late arrivals or no-shows
- Preventing double bookings of facilities
- Ensuring that max_participants_at_creation matches the facilities capacity when the group lesson is held
- Group lesson bookings have to made by someone with a sit membership
- Check whether the user is allowed to cancel a registration
- Group lesson arrival has to be after a user has arrived at the gym already
- All queue position logic
- Arrival times are within opening hours of the gym
- Group lessons are within the opening hours of the gym

Business logic only implemented by the application layer

- Check that all dates are valid

Questions:

1. Yes we can insure this. There can't be two group lessons with the same start time and instructor, because of group_lesson's primary key. The current model is not waterproof,

we only check start time, if we made bookings only possible in half hour segments we could implement this fully using only basic SQL logic, but this would require redesigning large parts of schema, making bookings much less flexible and making registrations much more cumbersome to manage (as they would have to be registered for all half hour segments).

2. Yes, with the UNIQUE constraint on group_lesson_registration on (person_id, start_time) we ensure that no person is able to register themselves to two group lessons that start at the same time. However, it doesn't check whether or not a person registered for a different group lesson that starts a few minutes later without the first one ending; this has to be fixed by triggers, the application layer or by making bookings half-hour specific as in task 1.
3. Use case number 6 needs blacklisting as the user has received three dots within 30 days.
4. That depends on the query, certain entities store historical data (e.g. group lessons max_participants_at_creation), but the current model is vulnerable to changes. For example, If extra bikes are added to a facility, that might change a "bikes per person" statistic for a specific group lesson instance. To hinder this loss of data either some snapshot functionality would have to be implemented, or historical data would have to be stored explicitly. Some statistics can be collected without difficulty because the information does not change over time and does not depend on other (changeable) data. For example, determining the total number of spinning classes held during a given period is straightforward and does not require storing additional information.